
Module 3, Chapter 3
Handling Limitations: Memory, Bias, and Resetting

LLMs are powerful tools. They're also genuinely limited in ways that aren't always obvious — and the limitations don't announce themselves. A model won't tell you it's forgotten the beginning of your conversation, or that its training data skews a particular direction, or that it's confidently filling in a gap it doesn't actually know the answer to. Understanding these three patterns — memory, bias, and when to reset — is what separates people who use LLMs effectively from people who get burned by them.

Memory: what the model actually remembers

Here's the thing most people don't realize: LLMs have no persistent memory. Every conversation starts from zero. When you open a new chat, the model has no idea who you are, what you worked on last week, or what you told it an hour ago in a different window.

Within a single conversation, the model can reference everything that's been said — but only up to its context window limit. Think of the context window as a spotlight: everything inside it is visible, everything outside it doesn't exist. As a conversation gets longer, older parts fall out of view. That's why a model might contradict something it said twenty messages ago — it literally can't see that far back anymore.

What to do about it:

Summarize and restate at the start of long sessions. If you're working on something complex across multiple turns, periodically drop in a quick summary: "To recap: we're building a customer support bot for a fintech company, the tone should be professional but approachable, and we've decided against using bullet points in responses." That keeps the model oriented even as the conversation grows.

For recurring tasks, keep a prompt file. A short paragraph describing your project, your preferences, and any decisions already made — pasted at the start of a new session — is faster and more reliable than hoping the model remembers. Try it in Groq (or your preferred LLM) with any project you return to regularly.

Bias: where it comes from and what to watch for

The model learned from human-generated text. Human-generated text contains human biases — about gender, profession, geography, culture, and a hundred other dimensions. The model absorbed all of it, and some of it shows up in outputs in ways that aren't always obvious.

This isn't a flaw unique to LLMs. Any system trained on human data inherits human patterns. But it's worth knowing where to look.

Common places bias surfaces:

Default assumptions about roles. Ask for a story about a surgeon and a nurse, and many models will default to making the surgeon male and the nurse female — unless you specify otherwise. The fix is simple: be explicit about the details that matter to you.

Geographic and cultural defaults. Ask for "a typical family dinner" and you'll likely get something that reflects Western, English-speaking norms. Ask for "a typical family dinner in Lagos" or "in rural Japan" and you get something completely different. The model isn't wrong in the first case — it's just defaulting to what was most represented in its training data.

Recency gaps. The model has a knowledge cutoff. Events after that date don't exist as far as it's concerned — and it won't always tell you that. If you're asking about anything that could have changed recently — company leadership, product releases, legislation, research findings — verify independently.

Confident fabrication. This is the one that catches people most off guard. Models will sometimes produce specific-sounding facts — statistics, citations, dates, names — that are plausible but wrong. The model isn't lying. It's pattern-matching, and sometimes the pattern produces something that sounds right without being right. For anything high-stakes, check the source.

What to do about it:

Be specific about what you want. Specify the demographic details that matter. Ask for sources and verify them. And for factual claims in anything consequential — a client document, a legal summary, a medical explanation — treat the model as a first draft, not a final answer.

Resetting: when to start fresh

Sometimes the right move is to stop iterating and start over. A few signals that you've hit that point:

The conversation has drifted. You started asking about email copy and somehow ended up in a long thread about brand voice guidelines. The model is now so deep in that context that it's hard to get a clean answer to a simple question. Start a new session, paste only the context that's relevant, and ask again.

The model is stuck in a pattern. If you've asked for the same thing three different ways and keep getting a variation of the same wrong answer, the model has anchored on something in the conversation that's pulling it off course. A fresh start with a better initial prompt will outperform another round of iteration.

The context window is getting long. In a very long conversation, the model's attention gets diluted. It may start producing vaguer, less focused responses as it tries to hold too much context at once. If the quality is dropping, a fresh session with a summary of where you are will usually recover it.

You need a different perspective. If you've been developing an idea in conversation with a model, that model has been shaped by everything you've already said. A fresh session — or a different model entirely — will approach the same question without those accumulated assumptions. Sometimes that's exactly what you need.

Putting it together

Memory, bias, and knowing when to reset aren't problems to solve — they're constraints to work with. The model doesn't know what it doesn't know. It won't flag its own blind spots. That's your job. And once you're aware of these patterns, you'll start noticing them quickly and correcting for them just as fast.

The good news: most of the mitigations are simple. Restate context. Be specific. Verify facts. Start fresh when you're stuck. None of these take long. All of them make a real difference.

That's a wrap on Module 3. In Module 4 we move from prompt engineering into the other half of this course — Retrieval-Augmented Generation. You'll learn why plain LLMs aren't enough for many real-world applications, and how RAG systems solve the problems we've been working around.